Module 5 — MCP Servers

Lesson 5.3

Build a Python MCP server

Use the mcp Python package. Define tools and resources with decorators. Run over stdio. About 50 lines for a working server.

Why it matters

If an off-the-shelf server doesn't cover your case (or you want a custom view of your own systems), writing one is small and rewarding. The Python SDK is the fastest path.

This lesson walks through what's in examples/04-mcp-server-python — a server that exposes a folder of markdown notes as searchable resources plus a `search_notes` tool.

Prereqs

- Python 3.11+
- `pip install "mcp[cli]"` (or use `uv add mcp` if you use `uv`)

Minimal server

```
# server.py
from pathlib import Path
from mcp.server.fastmcp import FastMCP

NOTES_DIR = Path.home() / "notes"

mcp = FastMCP("notes-server")

@mcp.tool()
def search_notes(query: str) -> list[dict]:
    """Search notes for the given query. Returns matching filename and a snippet."""
    results = []
    for path in NOTES_DIR.rglob("*.md"):
        text = path.read_text(encoding="utf-8", errors="ignore")
        if query.lower() in text.lower():
            idx = text.lower().find(query.lower())
            snippet = text[max(0, idx - 60): idx + 120].replace("\n", " ")
            results.append({"path": str(path.relative_to(NOTES_DIR)), "snippet": snippet})
        if len(results) >= 10:
            break
    return results

@mcp.resource("note://{path}")
def get_note(path: str) -> str:
    """Get the full content of a note by relative path."""
    full = NOTES_DIR / path
    if not full.is_relative_to(NOTES_DIR):
        raise ValueError("path escapes notes dir")
    return full.read_text(encoding="utf-8")

if __name__ == "__main__":
    mcp.run()
```

That's a complete MCP server. It:

- Exposes one tool (`search_notes`) callable by Claude with a query string
- Exposes resources under the `note://` scheme that return note contents
- Validates path inputs to prevent escape attacks

Wire it into Claude Code

In your `.claude/settings.json`:

```
{
  "mcpServers": {
    "notes": {
      "command": "python",
      "args": ["/abs/path/to/server.py"]
    }
  }
}
```

Restart `claude`. Confirm:

> *"List MCP tools. Is there a `search_notes` tool?"*

Try:

> *"Search my notes for 'kubernetes'. Then read the most relevant one in full and summarize."*

Key API patterns

Tools

```
@mcp.tool()
def my_tool(arg1: str, arg2: int = 0) -> str:
    """The docstring becomes the tool description Claude sees."""
    ...
```

- The function's docstring is the tool description (this is what Claude reads to decide whether to call). Write it for Claude, not for humans — be precise about what the tool does and when to use it.
- Type hints become the tool's parameter schema. Use them.
- Return Python primitives, lists, or dicts — they're serialized to JSON automatically.

Resources

```
@mcp.resource("scheme://{var}")
def my_resource(var: str) -> str:
    ...
```

- The URI pattern is templated; `{var}` becomes a function argument.
- Resources return strings (typically) or structured data.
- Use a clear scheme — `note://`, `db://`, `internal://`. Don't conflict with built-in schemes.

Prompts (templates)

```
@mcp.prompt()
def code_review_prompt(file_path: str) -> str:
    """A prompt template for reviewing a specific file."""
    return f"Review the file at {file_path}. Focus on correctness and style."
```

Prompts are server-defined named templates Claude can be asked to use. Less common than tools/resources but powerful for standardized workflows.

Logging and debugging

The server's stdout is the MCP wire protocol — don't `print()` there. Use stderr for logs:

```
import sys
print("server started", file=sys.stderr)
```

Or use the `logging` module configured to stderr.

When debugging, run the server manually first:

```
python server.py
# (sits waiting on stdin)
```

If that runs without errors, the issue is in your Claude config, not your server.

Lifecycle

The server boots when `claude` starts and dies when `claude` exits. If you need long-lived state (a DB connection, a cache), initialize it lazily in your tool functions — first-call cost is fine; per-call cost is not.

```
_conn = None
def get_conn():
    global _conn
    if _conn is None:
        _conn = psycopg.connect(os.environ["DATABASE_URL"])
    return _conn
```

Try it

1. Clone or create examples/04-mcp-server-python.
2. Point `NOTES_DIR` at a folder you have markdown in (or create some).
3. Wire it into `settings.json`.
4. Restart `claude`. Search and read notes through MCP.
5. Add a tool: `delete_note(path: str)`. Be deliberate about whether you want it default-allowed.

Gotchas

- **Don't `print()` to stdout.** Stdout is the MCP transport. Use stderr for everything human-readable.
- **Async vs. sync.** The decorators work for both. If you do I/O, you may prefer `async def` — check the SDK docs for your version.
- **Errors raised in tools propagate to Claude.** Make error messages informative — Claude will try to recover or report based on them.
- **Resource URIs must be valid URIs.** Spaces and weird characters need encoding. Stick to simple slugs.

Further reading

- Official Python MCP SDK: https://github.com/modelcontextprotocol/python-sdk
- examples/04-mcp-server-python — full runnable
- 5.4 Resources, tools, prompts — choose the right primitive

Next

→ 5.4 Resources, tools, and prompts